

Claude Code in VS Code — Complete Guide & Best Practices

Installation

Prerequisites

- VS Code version **1.98.0 or higher**
- An Anthropic account (claude.ai login)

Install the Extension

1. Press `Ctrl+Shift+X` to open Extensions
2. Search **"Claude Code"**
3. Click **Install**

Or open this in your browser: `vscode:extension/anthropic.claude-code`

If the extension doesn't appear after installation, run **"Developer: Reload Window"** from the Command Palette (`Ctrl+Shift+P`).

First-Time Setup

1. Click the **Spark icon** (⚡) in the top-right corner of the editor (appears when a file is open)
2. Click **Sign in** and complete browser authorization
3. Run **"Claude Code: Open Walkthrough"** from Command Palette for a guided tour

Key Shortcuts

Action	Windows/Linux
Open Extensions	<code>Ctrl+Shift+X</code>
Open Command Palette	<code>Ctrl+Shift+P</code>
Open Integrated Terminal	<code>Ctrl+`</code>
Open new terminal tab	<code>Ctrl+Shift+`</code>
Insert @-mention of selected code	<code>Alt+K</code>

Action	Windows/Linux
Multi-line input (no send)	Shift+Enter
Switch Plan Mode / Normal Mode	Shift+Tab
Expand/collapse all thinking blocks	Ctrl+0

Windows setup tip: Run `/terminal-setup` inside your first Claude Code session — it auto-configures the Shift+Enter keybinding for multi-line prompts.

Permission Modes

Switch modes by clicking the mode indicator at the bottom of the prompt box.

Mode	Behaviour
Normal	Claude asks permission before each action
Plan	Claude describes what it will do, waits for your approval before making any changes
Auto-accept	Claude makes edits without asking — use only on familiar codebases

Best Practice 1 — Set Up CLAUDE.md First

Before starting any project, run `/init` inside a Claude session. It auto-generates a `CLAUDE.md` by analysing your codebase. This is Claude's **long-term project memory** — it reads this every session.

What to Include

md

Architecture

- Frontend: Angular + TypeScript
- Backend: .NET API

Code Standards

- Standalone components only, no NgModules
- inject() over constructor injection
- Signals over BehaviorSubject
- Named exports over default exports

Key Files

- Services: /src/app/services/
- Components: /src/app/components/
- Types: /src/app/models/

Commands

- Dev: ``ng serve``
- Tests: ``ng test``
- Build: ``ng build``
- Lint: ``ng lint``

Without CLAUDE.md, Claude starts every session with no project context and makes generic decisions.

Best Practice 2 — Always Use Plan Mode Before Coding

Never let Claude jump straight into writing code. Use the **Explore → Plan → Execute** workflow.

The Three-Phase Workflow

Phase 1 — Explore

Act as a Senior Architect. Before proposing any implementation:

1. Analyse the existing codebase in /src
2. Identify all files affected by this change
3. List integration points and dependencies
4. Note architectural constraints

Phase 2 — Plan

Draft a detailed implementation plan. Include:

- Exact file paths for new/modified files
- Function signatures that need changes
- Testing strategy

Present as a numbered checklist. Do NOT write code yet.

Phase 3 — Execute Only after you've reviewed and approved the plan above.

Anthropic's internal testing found unguided attempts succeed ~33% of the time. A reviewed plan gets it close to 100%.

Best Practice 3 — Use @-Mentions Instead of Copy-Pasting

- Press `(Alt+K)` to insert an @-mention of your currently selected code (e.g. `@component.ts#12-45`) into your prompt
 - Reference specific line ranges — Claude reads exactly what you mean, nothing more
 - Faster and more precise than describing or copying code manually
-

Best Practice 4 — Review Every Diff Before Accepting

- Claude shows a **side-by-side diff** of original vs proposed changes before editing any file
 - You can **accept**, **reject**, or **edit the proposed content directly** in the diff view
 - If you edit the diff before accepting, Claude is told you modified it
 - Never use auto-accept mode on an unfamiliar codebase
-

Best Practice 5 — Use Sub-Agents for Specialised Tasks

One Claude session for everything causes context pollution. Use `/agents` to create focused agents.

Useful Agent Prompts

Debugger Agent

You are the Debugger Agent. Only identify and fix bugs.

When given an error: analyse the stack trace, identify root cause, propose the minimal fix, and explain why it occurred.

Security Agent

Review code exclusively for security vulnerabilities.
Flag: SQL injection, XSS, auth flaws, data exposure, insecure dependencies.

Architecture Agent

You are a technical architect. Create implementation plans, not code.
Focus on patterns, scalability, and maintainability.

Best Practice 6 — Extend Claude with MCP Servers

Inside a Claude session, run `/mcp` to configure external tool access.

Recommended MCP Servers

Server	Use
context7	Up-to-date framework docs (Angular, .NET, etc.)
GitHub	Read issues, PRs, and repo context
Playwright	Browser automation and e2e testing
Figma	Read design files and components

For Angular development, **context7** is the most valuable — gives Claude current Angular docs instead of relying on potentially outdated training data.

Browse more: <https://registry.modelcontextprotocol.io>

Best Practice 7 — Manage Context Window Actively

The core problem: Every file Claude reads, every command output, every message eats into the context window. When it fills, Claude starts forgetting earlier instructions.

The Context Indicator

The prompt box shows how much context is used. Watch it actively.

Strategies to Avoid Exhausting the Token Limit

Strategy	What to Do
Use <code>/compact</code> regularly	Run <code>/compact</code> manually before context gets critical — Claude summarises the session and continues
One task per session	Don't mix debugging + feature building + refactoring in one long session
Reference files with @-mentions	Instead of pasting large code blocks into the chat, use <code>@filename#line-range</code>
Keep <code>CLAUDE.md</code> tight	A concise <code>CLAUDE.md</code> (under 150 lines) loads faster and leaves more context for actual work
Use Plan Mode for large tasks	Claude explores and plans with less back-and-forth, fewer wasted tokens
Avoid re-explaining context	If you've set up <code>CLAUDE.md</code> correctly, you never need to re-explain the project in each message
Start fresh sessions for new features	Don't carry a debugging session into a new feature — open a new session, <code>CLAUDE.md</code> loads automatically
Avoid large file dumps	Never paste entire files into the prompt — @-mention specific line ranges instead
Don't ask Claude to re-read files	Claude already reads files it edits — only @-mention when you want it to read something specific
Run <code>/compact</code> before context warning	Don't wait for the warning — compact proactively around 60–70% context usage

Signs You're Running Low on Context

- Claude starts giving generic answers that ignore your project conventions
- It contradicts earlier decisions in the same session
- It suggests solutions that don't match your stack
- Responses become shorter and less detailed

When any of these happen: run `/compact` or start a new session.

Best Practice 8 — Use Extended Thinking for Complex Problems

Toggle **Extended Thinking** via the command menu (`/`) for tasks like:

- Architecture decisions
- Debugging hard-to-trace bugs
- Refactoring large modules
- Performance optimisation analysis

Claude's reasoning appears as collapsible blocks — press `Ctrl+0` to expand/collapse all.

Quick Reference — Commands Inside Claude Code

Command	What it Does
<code>/init</code>	Auto-generate CLAUDE.md from your codebase
<code>/compact</code>	Summarise session to free up context
<code>/mcp</code>	Configure MCP servers
<code>/agents</code>	Create specialised sub-agents
<code>/usage</code>	View plan usage
<code>/terminal-setup</code>	Configure Shift+Enter for multi-line input
<code>/remote-control</code>	Start a Remote Control session

For Angular Development Specifically

- Set up CLAUDE.md with your Angular version, standalone component rule, and Signal-first convention
- Use Plan Mode before creating any new component, service, or route
- @-mention specific component or service files when asking for refactors
- Create a **Debugger Agent** for RxJS/async issues — keeps those long stack traces out of your main session
- Add **context7** MCP so Claude always has current Angular docs

Last updated: May 2026